TIKO — Tech Stack

Oct 8, 2026 · @ki

TIKO is built as one Next.js and TypeScript codebase backed by PostgreSQL, which is the fastest path for a solo developer and replaces the separate NestJS API and BullMQ queue from the PRD.

1. Stack at a glance

Layer	Choice	Why
Framework	Next.js (App Router) + TypeScript	One repo for UI and backend via route handlers and server actions
Styling	Tailwind CSS + shadcn/ui	Matches the pills, 24 px cards and CSS tokens in the Design Doc
Font and icons	Nunito via `next/font`; Lucide	Both named in the Design Doc
Animation	Framer Motion	Ring fills, springs, streak dot pop, confetti
Theming	`next-themes`	Light and dark from the same variables
Client state	TanStack Query	Caching and optimistic updates for instant feedback
Offline storage	Dexie (IndexedDB)	Logs queue offline and sync later
PWA	Serwist	Service worker, home-screen install, web push
Database	PostgreSQL on Neon or Supabase	Relational data suits goals, logs, budgets; free tiers
ORM	Drizzle (or Prisma)	Typed schema and migrations
Validation	Zod	One schema for forms and API
Auth	Auth.js (magic link + Google) or Supabase Auth	No passwords to store

Layer	Choice	Why
Goal engine	A TypeScript module called after every log	Treats unlock in the same request
Scheduled jobs	Vercel Cron or Supabase cron	Nightly streak grace, Sunday recap
Notifications	`web-push` + Resend (email)	Email is the fallback when push fails
Payments (V1 Plus)	Razorpay	INR subscriptions
Hosting	Vercel	Zero-ops deploys and CDN
Monitoring and analytics	Sentry; PostHog (events only)	Never send journal text or exact amounts
Testing	Vitest; Playwright	Goal engine unit tests; end-to-end log, goal, treat flow

2. Architecture

The browser only talks to the Next.js server; the server owns the database, the goal engine and all notifications.

1. **Browser (PWA):** React UI, TanStack Query cache, Dexie queue for offline logs, service worker for install and push.
2. **Next.js server on Vercel:** route handlers and server actions validate input with Zod, write to PostgreSQL through Drizzle, then call the goal engine in the same request.
3. **Goal engine (a module, not a service):** after each log it recomputes progress, completes goals, unlocks treats and awards XP, so the celebration appears instantly.
4. **PostgreSQL:** all user data; journal bodies are encrypted by the server before storage.
5. **Scheduled jobs (cron):** nightly day close and streak grace rule; Sunday weekly recap; reminder sends.
6. **Notifications:** `web-push` first, Resend email as fallback, never more than 2 a day.
7. **Auth service:** magic link and Google; sessions are checked in server actions.

If load or job complexity ever outgrows route handlers, the goal engine and jobs can move into a separate Node service without changing the browser code.

3. Key technical decisions

Topic	Decision	Reason
Offline logging	Write to Dexie first, update the UI optimistically, sync in the background; latest edit wins per entry	Log feedback in under 100 ms, works with no connection
Goal engine	Pure functions over logs (`evaluateGoal(goal, logs)`) with unit tests	Easy to test; no hidden state
Streak grace	Computed from daily totals: one miss per 7 days is forgiven, two in a row break	Matches the PRD rule exactly
Money	Store integer paise, format on display	Avoids floating-point errors
Journal privacy	AES-GCM encryption of `body` on the server; key in an environment secret	Encrypted at rest without extra infrastructure
Guest mode	Data in Dexie until signup, then merged to the account	Zero-friction first use
Timezones	Store user timezone; "today" is computed per user	Daily rings and reminders match local days
Analytics	Event names and counts only	Keeps journal text and amounts out of PostHog
Accessibility	Radix primitives (via shadcn/ui), 48 px targets, reduced-motion check	WCAG 2.2 AA target

4. Project setup

Start with `npx create-next-app@latest tiko --typescript --tailwind --app`, then add the packages below.

```
npm i drizzle-orm zod @tanstack/react-query dexie framer-motion next-themes
lucide-react web-push
npm i -D drizzle-kit vitest @playwright/test
npx shadcn@latest init
```

Folder structure

```
tiko/
  app/
    (tabs)/today  goals  money  mind  me
    api/ (cron, push)
  components/ (habit-card, progress-ring, streak-chain, goal-card, treat-card, quick-add-sheet, mood-picker)
  lib/
    db/ (schema.ts, client.ts)
    goal-engine/ (evaluate.ts, streaks.ts, xp.ts)
    offline/ (queue.ts, sync.ts)
    crypto.ts
  styles/tokens.css
  tests/
```

TIKO tokens in Tailwind (colors come from the Design Doc; use the CSS variables so dark mode swaps them in one place):

```
// tailwind.config.ts (excerpt)
theme: { extend: {
  colors: {
    cream: 'var(--cream)', paper: 'var(--paper)', mist: 'var(--mist)',
    ink: 'var(--ink)', slate: 'var(--slate)',
    lagoon: { DEFAULT: 'var(--lagoon)', mist: 'var(--lagoon-mist)' },
    habits: { DEFAULT: 'var(--habits)', deep: 'var(--habits-deep)', tint: 'var(--habits-tint)' },
    money: { DEFAULT: 'var(--money)', deep: 'var(--money-deep)', tint: 'var(--money-tint)' },
    mind: { DEFAULT: 'var(--mind)', deep: 'var(--mind-deep)', tint: 'var(--mind-tint)' },
    goals: { DEFAULT: 'var(--goals)', deep: 'var(--goals-deep)', tint: 'var(--goals-tint)' },
  },
  borderRadius: { card: '24px', pill: '999px' },
  boxShadow: { card: '0 8px 24px rgba(46,58,70,0.08)' },
  fontFamily: { sans: ['var(--font-nunito)', 'system-ui', 'sans-serif'] },
}}
```

5. Build order for the 8-week MVP

Build the log, goal, treat loop first and add modules after it works; estimates assume one full-time developer, so stretch them if you build part-time.

Weeks	Milestone	Done when
1	Setup: Next.js, Tailwind tokens, Nunito, database schema, auth	You can sign in and see an empty Today screen
2 to 3	Habits and Today screen: habit cards, daily ring, offline queue	Tap logs a unit in under 100 ms, even offline
4	Goal engine, treats, celebration screen, Trophy shelf	Completing a goal shows the treat and Claim works
5	Money: quick add, monthly budget, safe-to-spend	Expense in two taps; undo toast works
6	Mind: mood check-in, simple journal with encryption	Journal body is unreadable in the database
7	Streak grace, reminders, PWA install, weekly recap job	Push and email reminders arrive on schedule
8	Accessibility pass, dark mode, Sentry, beta with 10 users	Contrast and tap-target checks pass; beta users log 4+ days a week